AerialHDMapper: High-Definition Map Construction From Aerial Imagery

Haofeng Xie, Xiangyun Hu, Huiwei Jiang, Hengming Dai, Pengwei Dong, and Pengfei Wang

***Abstract*— Lane-level high-definition map (HDMap) automatic construction from aerial imagery can significantly enhance the efficiency of HDMap production. However, some problems limit the application of aerial imagery in HDMap construction. First, the bird's eye view of aerial imagery often causes occlusion, which leads to interrupted map element predictions. In addition, existing methods for aerial imagery-based HDMap construction can only construct road-level map and the postprocessing is complicated. To solve these problems, we propose AerialHDMapper, an end-to-end framework that predicts lane-level vectorized map elements directly requiring no postprocessing. The framework improves prediction accuracy in occluded areas through the introduction of the directional attention (DirAttn) module. AerialHDMapper is mainly composed of two parts: the image encoder and the polyline generator. The image encoder integrates DirAttn modules, which leverage information along and perpendicular to the direction of a map element, thereby improving the representation of the current position. The polyline generator employs a modified transformer architecture to identify long-sequence irregular map elements. Experimental results demonstrate that AerialHDMapper outperforms existing methods on both the CARLA Simulator Dataset and the Aerial Argoverse2 Dataset we collected. Extensive ablation experiments further reveal that our approach exhibits strong robustness and generalization capabilities.**

***Index Terms*— Aerial imagery, attention, high-definition map (HDMap), map construction, transformer.**

I. Introduction

THE high-definition map (HDMap) [1] is a detailed representation that provides critical information for autonomous driving, such as lane dividers, lane centerlines, and road geometry. This information enables autonomous vehicles to navigate safely and make appropriate decisions [2], [3], [4]. Traditionally, HDMaps are manually annotated using data from multiple sensors including light detection and ranging (LiDAR), global positioning system (GPS), inertial measurement units (IMUs), and vehicle motion sensors [5], [6], [7], [8]. However, it is a labor-intensive and expensive process and incurs high maintenance costs. To tackle these issues, deep learning-based automatic extraction methods have gradually become mainstream by extracting information from aerial imagery [9], [10], [11], onboard images [12], [13], [14], and LiDAR [15]. These methods have significantly streamlined the HDMap production pipeline and reduced maintenance costs.

Existing aerial imagery-based HDMap construction studies primarily focus on road-level maps. Representative methods can be divided into two categories: segmentation-based and graph-based methods. Segmentation-based methods [11], [16], [17] consider HDMap extraction as a semantic segmentation task. These methods employ semantic segmentation networks to classify pixels and then vectorize the results through postprocessing techniques, such as thinning [18], [19]. Graph-based methods [10], [20], [21] directly generate the graph that represents target objects using an iterative graph growing process. Starting from an initial point, these methods determine the position of the next point based on image features, gradually forming map elements.

Few studies focus on aerial imagery-based lane-level HDMap construction. MarkCapsNet [22] tries to extract road markings from aerial imagery. He and Balakrishnan [9] predict lane centerlines in the intersection and nonintersection areas separately. However, due to the problems of occlusion and low resolution in aerial images, lane-level HDMap construction methods [13], [14] mainly take on-board images as data sources. These methods predict bounding boxes and then predict polylines within those bounding boxes to achieve lane-level construction results.

Despite achieving promising results, current aerial imagery-based HDMap construction methods face inherent limitations. First, the "what you see is what you get" nature of segmentation-based methods results in poor performance when images contain occlusions or categories that lack physical annotations, as illustrated in Fig. 1. This drawback significantly hinders the practical effectiveness of segmentation-based methods. Second, neither segmentation-based nor graph-based methods operate as end-to-end frameworks. The absence of an integrated approach makes extracting instance-level predictions cumbersome and time-consuming. These limitations make lane-level prediction particularly challenging for aerial imagery-based frameworks.

To solve the aforementioned problems, we propose AerialHDMapper, an end-to-end framework that leverages the

Received 16 August 2024; revised 5 December 2024; accepted 24 January 2025. Date of publication 10 February 2025; date of current version 21 February 2025. This work was supported in part by the Special Funds of Hubei Luojia Laboratory under Grant 220100028 and Grant 230700006; and in part by the Fundamental Research Funds for the Central Universities, China, under Grant 2042022dx0001. *(Corresponding author: Xiangyun Hu.)*

Haofeng Xie, Pengwei Dong, and Pengfei Wang are with the School of Remote Sensing and Information Engineering, Wuhan University, Wuhan 430079, China (e-mail: xiehaofeng@whu.edu.cn; dongpw@whu.edu.cn; wangpf@whu.edu.cn).

Xiangyun Hu is with the School of Remote Sensing and Information Engineering, Wuhan University, Wuhan 430079, China, and also with Hubei Luojia Laboratory, Wuhan 430079, China (e-mail: huxy@whu.edu.cn).

Huiwei Jiang is with the National Geomatics Center of China, Beijing 100044, China (e-mail: jianghw@ngcc.cn).

Hengming Dai is with the Institute of International Rivers and Eco-Security and the School of Earth Sciences, Yunnan University, Kunming 650500, China (e-mail: daihm@ynu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2025.3540439

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

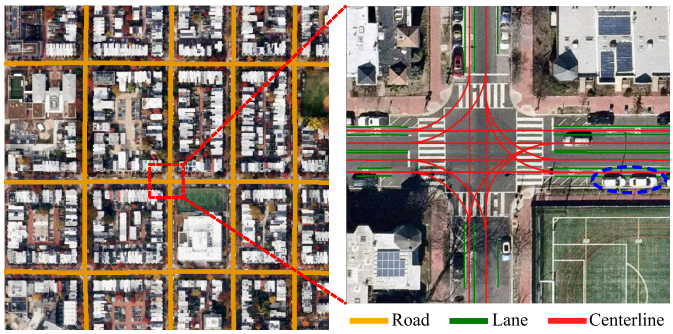


Fig. 1. Relationship between road-level and lane-level HDMap. The road-level HDMap on the left provides a general overview, while the lane-level HD map on the right offers detailed annotations of lanes and centerlines. The blue dashed circle highlights the occlusion caused by shadows.

directional attention (DirAttn) module to recover information lost due to occlusions. This novel method directly produces vectorized map elements without requiring complex post-processing or slow iterative growing processes. Specifically, AerialHDMapper adopts an encoder–decoder structure. The encoder is a modified ResNet [28] with DirAttn modules inserted between layers. Inspired by the manual annotation process, where occluded map elements are filled in using along-line or perpendicular information, the DirAttn module performs attention operations along the map element direction and its perpendicular direction. Motivated by DETR [29], we utilize a transformer-based decoder to identify map elements. Unlike DETR, which predicts bounding boxes, AerialHDMapper predicts long-sequence irregular polylines as map elements. Moreover, the semantic segmenter (SemSeg) is also included for multitask learning to enhance prediction accuracy and filter out inaccurate vector results.

We critically examine the proposed method on two datasets we collected: 1) CARLA Simulator Dataset, a simulated aerial HDMap dataset sampled from the CARLA simulator [30] and 2) Aerial Argoverse2 Dataset, a high-resolution aerial imagery HDMap dataset that we collected according to the annotation of the Argoverse2 Dataset [31]. Experimental results demonstrate that AerialHDMapper achieves state-of-the-art (SOTA) performance on the aerial imagery-based HDMap construction task. In addition, the promising experimental results under different settings highlight the robustness of our model.

The main contributions of our work can be summarized as follows.

1) AerialHDMapper is the first end-to-end lane-level HDMap construction framework for aerial imagery. Unlike previous methods, we directly predict vectorized map elements without postprocessing or iterative growing processes.
2) The DirAttn module is proposed to solve occlusions and improve predictions for categories lacking physical annotations, such as lane centerlines.
3) The CARLA Simulator Dataset and the Aerial Argoverse2 Dataset we collected enrich the datasets for aerial imagery-based lane-level HDMap construction task.
4) We demonstrate that AerialHDMapper achieves SOTA performance on both datasets. Extensive ablation studies validate the effectiveness of its components.

II. Related Work

HDMap construction has attracted much attention as an essential part of autonomous driving. Traditionally, this process involves manual annotation based on LiDAR, GPS, IMU, and other data, making it laborious and costly. However, with the development of deep learning, numerous automatic HD Map extraction methods have emerged. These methods can be classified based on their data source: aerial imagery-based and vehicle sensor-based. Aerial imagery-based methods focus on road-level information, such as the road network, while onboard sensor-based methods focus on extracting lane-level information, such as lane lines, centerlines, and sidewalks. The summary of the different methods is shown in Table I.

A. Mapping With Aerial Imagery

Aerial imagery-based HDMap construction methods mainly focus on road-level information. These methods include segmentation-based and graph-based methods.

Mnih and Hinton [17] introduced the first road segmentation model with iterative refinement utilizing aerial images. Deeproadmapper [11] introduces an algorithm to connect the missing parts of segmentation results. Batra et al. [16] proposed a connectivity task predicting road mask orientation. Subsequently, the connectivity refinement network iteratively enhances the estimated road networks using orientation and segmentation predictions. He and Balakrishnan [9] validate all possible turning lanes to extract the valid turning lanes at intersections. Although segmentation-based methods are straightforward, their pixel-level prediction results require complex postprocessing, which is misaligned with downstream tasks.

Graph-based methods create the road graph through iterative graph growing, which generates vertices and the polyline object from an initial vertex. Roadtracer [10] proposes an iterative graph construction algorithm that utilizes a CNN-based decision function to generate a road network graph. DagMapper [20] focuses on predicting highway lane boundaries by modeling the problem as inference in a directed acyclic graphical model. Icurb [21] tries to let the agent iteratively detect road curb graphs using local feature maps. RNGNet and RNGNet++ [23], [32] construct the incremental graph through an autoregressive approach. Graph-based methods directly obtain vector results, which simplifies postprocessing. However, the iterative process makes them inefficient. In addition, only small patches are considered at a time, making graph-based methods challenging to identify objects with no physical markings, such as lane centerlines.

Our end-to-end transformer model delivers faster lane-level results without the need for complex postprocessing or iterative growing algorithms. Moreover, the proposed DirAttn module further enhances predictions for occluded objects and objects without physical markings.

B. Mapping With Vehicle Sensor

The construction of HD maps based on vehicle sensors focuses on learning lane-level HD maps with segmentation and vectorization methods. Segmentation-based methods

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF HDMAP CONSTRUCTION METHODS

Data Source	Map Level	Method Type	Examples	Characteristics
Aerial Imagery	Road level	Segmentation Methods	Mnih et al. [17] DeepRoadMapper [11]	Complex post-processing Pixel outputs
		Graph Methods	RoadTracer [10] DagMapper [20] ICurb [21] RNGNet [23]	Iterative process Vector outputs
Vehicle Sensors	Lane level	Segmentation Methods	HDMapNet [12] BEVFormer [24] UniFusion [25]	Complex post-processing Pixel outputs
		Vectorization Methods	VectorMapNet [13] MapTR [14] StreamMapNet [26] PivotNet [27]	End-to-end Vector outputs Unsuitable for remote sensing

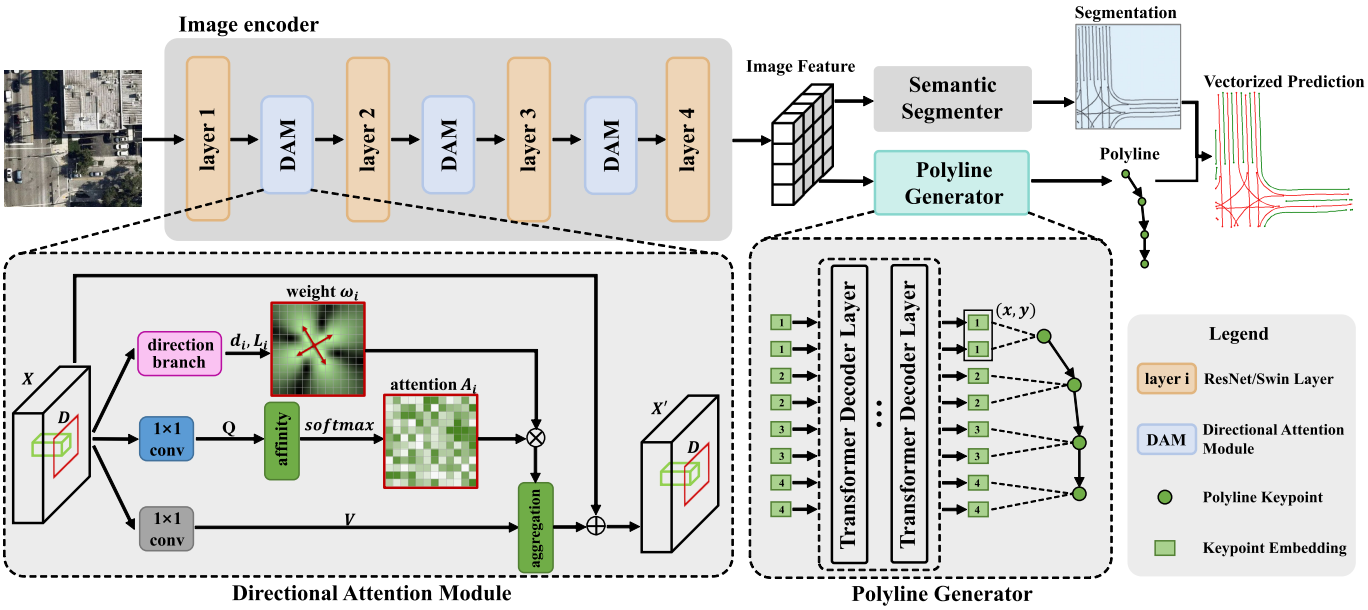


Fig. 2. Overview of AerialHDMapper. The image encoder transforms aerial imagery into image features. The DirAttn module is situated between the two layers of the image encoder. Using the image features, the SemSeg produces segmentation results, while the polyline generator generates the polyline result.

[12], [24], [25] require complicated postprocessing to provide HD map element polylines for downstream modules. The vectorization-based method was first proposed by VectorMapNet [13]. These methods [13], [14], [26], [27] predict elements as a sequence of points based on object detection. As Vectorization methods are end-to-end, they eliminate the need for complex postprocessing.

VectorMapNet introduces an end-to-end vectorized HD map learning pipeline that predicts sparse sets of polylines from sensor observations. MapTR [14] proposes a unified permutation-equivalent modeling approach to better describe the shape of map elements. StreamMapNet [26] uses temporal information as the memory buffer to empower the construction of HDMap.

Typically, vectorized methods locate the map element's bounding box and then use a polyline generator to identify the element keypoints based on the bounding box. However, bounding boxes are not suitable for large-scale remote sensing images and objects. Our proposed innovative structure can yield better results in predicting polylines from aerial imagery without relying on bounding boxes.

III. PROPOSED METHOD

The objective of our task is to extract vectorized HDMap elements, such as lanes and lane centerlines, from aerial imagery. Specifically, we aim to develop a model that can predict a set of map elements $\mathcal{V} = \{V_1, \ldots, V_N\}$. Each map element V_i is defined as $V_i = \{P_i, c_i\}$, where $P_i = \{p_{i,n} \in \mathbb{R}^2 | n = 1, \ldots N_p\}$ represents a polyline composed of an ordered set of N_p 2-D coordinates, and $c_i \in \mathbb{R}^{N_{\text{cls}}}$ is a one-hot encoded vector indicating the category of each element. This representation is widely employed in downstream tasks of autonomous driving.

The proposed framework consists of two key parts, i.e., the image encoder and the polyline generator, as shown in Fig. 2. First, aerial images are input into the image encoder to obtain the image features $\mathcal{F}$. The image encoder consists of four layers, with the DirAttn module inserted between two layers.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

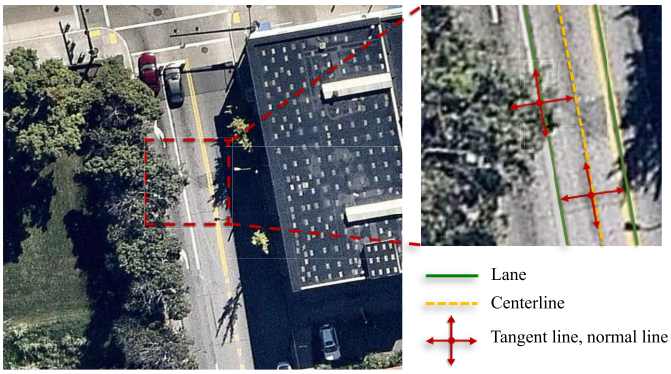


Fig. 3. Core idea of DirAttn module. Despite lanes being obscured by trees and centerlines lacking physical markings, their positions can still be inferred by analyzing the information in their tangential and orthogonal directions.

Then, the image features are passed to the polyline generator to predict the polyline coordinates P_i and the category c_i. The image features are also input into the SemSeg to predict the semantic segmentation results, which assist in predicting and postprocessing map elements.

A. Image Encoder

Our encoder is built based on ResNet [28] structure. According to the structure of ResNet, the network includes four stages. Each stage contains a specified number of bottleneck blocks. At the end of each stage, the DirAttn module is plugged in as shown in Fig. 2. It will be covered in detail in Section III-A.

DirAttn Module: Due to the bird's-eye perspective of aerial imagery, HDMap elements such as lanes are often occluded by obstructions like vegetation or buildings. In addition, centerlines, which are critical for detailed lane-level maps, lack physical representations in the imagery, making their prediction particularly challenging for standard neural networks.

To address these challenges, we propose the DirAttn module, which leverages contextual information from tangential and orthogonal directions around each point. By integrating directional context, our module can infer the presence and trajectory of HDMap elements even when they are occluded or visually absent (see Fig. 3). The DirAttn module fuses tangential and orthogonal information within a localized square patch, enhancing predictions while maintaining computational efficiency by focusing attention on relevant areas.

As shown in the bottom left of Fig. 2, the DirAttn module operates on a local feature map $X \in \mathbb{R}^{C\times H\times W}$. We apply two 1×1 convolutional layers to X to obtain feature maps $Q \in \mathbb{R}^{C'\times H\times W}$ and $V \in \mathbb{R}^{C\times H\times W}$, where C' is a reduced channel dimension.

We then generate the attention map $A \in \mathbb{R}^{D^2\times H\times W}$ by applying an affinity operation followed by a softmax function to Q. Specifically, the affinity operation utilizes an MLP layer to transform the channel dimension of Q from C' to D^2, where D is the patch size. The softmax normalizes this output into a probability distribution, representing attention between the current pixel and others within the patch.

Meanwhile, the direction branch is incorporated into the network to predict the normalized tangential direction $d_i \in [0, 1)$ at each spatial location (x_i, y_i). Specifically, the direction branch consists of multiple convolutional layers, and the sigmoid function is added at the end to obtain the normalized tangential direction. This branch learns directional information during training, allowing the network to identify the primary focus direction at each location.

Following this, the weight $\omega_i \in \mathbb{R}^{D^2\times 1}$ is calculated to capture the spatial correlation between pixel i and others within the patch region. Each weight $\omega_{i,j}$, the jth element of ω_i, represents the directional weight between pixel i and pixel j, and is calculated by considering both the spatial distance the spatial distance $L_{i,j}$ and the collinearity $\delta_{i,j}$

$$\omega_{i,j} = \frac{\delta_{i,j}}{\exp(\alpha L_{i,j})} \tag{1}$$

where the temperature parameter α controls the influence of distance—larger values of α reduce the contribution of distant pixels.

The spatial distance $L_{i,j}$ between current pixel i location (x_i, y_i) and the other pixel j location (x_j, y_j) is defined as

$$L_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{2}$$

where $L_{i,j}$ assigns higher weights to pixels closer to i.

The collinearity $\delta_{i,j}$ measures how aligned the direction from pixel i to pixel j is with the predicted direction d_i, and is defined as

$$\delta_{i,j} = |\cos(2(d_{i,j} - 2d_i\pi))| \tag{3}$$

$$d_{i,j} = \text{atan2}(x_i - x_j, y_i - y_j) + \pi. \tag{4}$$

The function atan2 is the two-argument arctangent, which returns the angle between the vector (x, y) and the positive x-axis, ranging from $-\pi$ to π. By computing $\delta_{i,j}$, we assign greater weight to pixels whose direction from i aligns with or is orthogonal to the predicted direction d_i.

Finally, we compute the output feature at position i using an aggregation operation

$$X'_i = X_i + (\omega_i \odot A_i)^T \text{box}_i(V) \tag{5}$$

where $X'_i \in \mathbb{R}^{1\times C}$ is the feature vector at pixel i in the output feature map $X' \in \mathbb{R}^{C\times H\times W}$. $A_i \in \mathbb{R}^{D^2\times 1}$ is the attention map at pixel i in A. $\odot$ denotes element-wise multiplication. The function $\text{box}_i(V)$ extracts features within the $D \times D$ patch centered at pixel i, reshaping them from $C \times D \times D$ to $C \times D^2$.

This design allows the DirAttn module to effectively capture directional context, enhancing the network's ability to predict lane lines and centerlines in aerial imagery, even in the presence of occlusions or absent visual cues.

B. Polyline Generator

1) Element Query: In the object detection task, existing transformer-based methods represent the bounding box of an object as a query embedding [29]. We extend this concept to HDMap elements $\mathcal{V}^{\text{poly}}$. We design the learnable element queries $Q^{\text{elem}} \in \mathbb{R}^{N_q\times d}$ as the input of polyline generator,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

where d is the embedding dimension and N_q is the number of element queries.

2) Architecture: The polyline generator is composed of a transformer decoder and a prediction head. The transformer decoder is built using multiple transformer layers, following the architecture proposed in DETR [29]. Each transformer layer includes multihead self-/cross-attention modules. In the self-attention module, attention operations are performed between queries, whereas in the cross-attention module, attention operations occur between image features and queries. In addition, to enhance computational efficiency, the deformable attention module [33] is employed as the cross-attention module.

The prediction head contains two MLP layers to get the point coordinates of elements and their categories based on elements queries Q^{elem}. The process of the prediction head can be formulated as follows:

$$c_i = \text{MLP}_{\text{cls}}(q_i^{\text{elem}}) \tag{6a}$$

$$P_i = \text{MLP}_{\text{pos}}(q_i^{\text{elem}}) \tag{6b}$$

where $q_i^{\text{elem}} \in \mathbb{R}^d, i = 1, \ldots, N_q$ is the ith element query of Q^{elem}. MLP_{cls} decodes q_i^{elem} into $c_i \in \mathbb{R}^{N_{\text{cls}}}$, where each dimension is the probability that the element query belongs to this category. MLP_{pos} decodes q_i^{elem} into $P_i \in \mathbb{R}^{2N_p}$. P_i is converted to the concatenation of normalized point coordinates $[x_1, y_1, \ldots, x_{N_p}, y_{N_p}]$, so that the MLP layer can predict it directly.

In addition, we use the results of the segmenter to improve the prediction results. We remove points that diverge from the location and category of the segmentation result. Specifically, if the map element points predicted by the polyline generator are too far away from the nearest segmentation pixel of the same category, this point will be deleted.

C. Matching Cost and Training Loss

AerialHDMapper employs an end-to-end training method. Once map elements are predicted by the polyline generator, they are paired with the ground truth elements based on the matching cost, addressing the issue that the number of predicted elements N_q is much larger than that of ground truth elements N. Subsequently, the polyline generator loss is calculated between the paired predicted elements and ground truth elements. Moreover, we set the segmentation loss to supervise the SemSeg. This section will provide a detailed explanation of the matching cost and the training loss function.

To pair predicted and ground truth map elements, we adopt the bipartite matching algorithm. Since $N_q \gg N$, we pad the set of ground truth elements with $\emptyset$ (background) up to N_q. Then, we compute the matching degree of the predicted and ground truth elements using the matching cost $\mathcal{L}_{\text{match}}$ and find the matching scheme σ^* with the minimum cost using the Hungarian algorithm [34].

For a predicted element $\hat{V}_i = \{\hat{P}_i, \hat{c}_i\}$ and a ground-truth element $V_i = \{P_i, c_i\}$, the matching cost is defined as follows:

$$\mathcal{L}_{\text{match}}\left(\hat{V}_i, V_i\right) = \mathcal{L}_{\text{poly}}\left(\hat{P}_i, P_i\right) + \mathcal{L}_{\text{Focal}}\left(\hat{c}_i, c_i\right). \tag{7}$$

Here, the focal loss $\mathcal{L}_{\text{Focal}}$ [35] is selected as the classification cost, and $\mathcal{L}_{\text{poly}}$ is the polyline cost. We compute smooth $L1$ loss [36] for all point permutations and select the minimum one as $\mathcal{L}_{\text{poly}}$. Specifically, $\mathcal{L}_{\text{poly}}$ is computed as follows:

$$\mathcal{L}_{\text{poly}}(\hat{P}_i, P_i) = \min_{\gamma \in \Gamma} \frac{1}{N_p} \sum_{j=1}^{N_p} \mathcal{L}_{\text{SmoothL1}}\left(\hat{p}_{\gamma(j)}, p_j\right) \tag{8}$$

where $\hat{p}_j$ is the jth point of $\hat{P}_i$. Γ is the permutation group of points introduced in MapTR [14].

To find the optimal matching pairs, we use the Hungarian algorithm which minimizes the total matching cost. This step can be described as

$$\sigma^* = \underset{\sigma \in \mathcal{P}}{\text{argmin}} \sum_{i=1}^{N_q} \mathbb{1}_{(c_i \neq \emptyset)} \mathcal{L}_{\text{match}}(\hat{V}_i, V_i) \tag{9}$$

where σ is a permutation of N_q predicted elements, and $\mathcal{P}$ is the permutation group of predicted elements. $\mathbb{1}_{(\hat{c}_i \neq \emptyset)}$ indicates that the value is 1 if the predicted category of element i is not the background $\emptyset$, and 0 otherwise.

After obtaining the permutation with the minimum matching cost σ^*, we can calculate the loss of the polyline generator as follows:

$$\mathcal{L}_{\text{gen}}(\hat{V}, V) = \sum_{i=1}^{N_q} \mathbb{1}_{(c_i \neq \emptyset)} \mathcal{L}_{\text{poly}}\left(\hat{P}_{\sigma^*(i)}, P_i\right) + \mathcal{L}_{\text{Focal}}\left(\hat{c}_{\sigma^*(i)}, c_i\right). \tag{10}$$

Finally, after adding the segmentation loss $\mathcal{L}_{\text{seg}}$ in the form of cross-entropy loss to supervise the SemSeg, the training loss can be defined as follows:

$$\mathcal{L}_{\text{train}} = \lambda_1 \mathcal{L}_{\text{gen}} + \lambda_2 \mathcal{L}_{\text{seg}}. \tag{11}$$

IV. Experiments

A. Dataset and Evaluation Metric

1) Aerial Argoverse2 Dataset: We create our dataset due to the limited availability of public datasets and resources for extracting HDMaps from aerial imagery. The HD map information within our dataset is sourced from the Argoverse2 [31] dataset, while the corresponding aerial images are gathered from Mapbox based on the HD map's geographic coordinates. As there is a deviation between the HD map in the Argoverse2 Dataset and the aerial imagery coordinates, we perform geo-registration between them to align them accurately.

The Aerial Argoverse2 Dataset covers HD maps in six cities, including Miami, Austin, Washington DC, Pittsburgh, Palo Alto, and Detroit. For the aerial images, we acquired them at zoomed-in view level 20 in Mapbox with a spatial resolution of 0.057 m per pixel. Regarding labels, we choose lane centerlines and lanes as the target categories. After obtaining the samples, we crop them into tiles of pixel size 768 × 768. The resulting dataset has 1775 tiles for training and 316 for testing.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

2) CARLA Simulator Dataset: Due to the scarcity of datasets for aerial imagery, many methods generate datasets based on the CARLA simulator, most of which are used for semantic segmentation [37], [38]. We are the first to use the CARLA simulator to simulate aerial images paired with vectorized map elements. Specifically, we simulated map scenes with a set number of vehicles driving randomly while adhering to traffic rules. Then, we randomly selected a vehicle from this set and placed a bird's-eye-view camera on it to capture images and the corresponding HDMap information. Once we collected the required number of images, we repeated this process with different vehicles to gather additional data. This method ensures a diverse and comprehensive dataset for evaluating our proposed method.

The dataset includes seven map scenes, and we randomly added various weather conditions to simulate a natural environment. The target categories in the CARLA Simulator Dataset are centerlines and lanes. As a result, we collected 1770 training tiles and 300 test tiles of size 1024 × 1024.

3) Evaluation Metric: Since our prediction results are in vector form, traditional pixel-level metrics like intersection-over-union (IoU) are not suitable for evaluation. Instead, we employ chamfer average precision (AP) as our evaluation metric. The chamfer AP is a distance-based metric that measures the similarity between predicted and ground-truth curves. Specifically, chamfer AP represents the average precision, with positive and negative samples determined by the Chamfer distance. Chamfer distance evaluates the similarity between two unordered sets by considering the distance of each permutation of the elements of the set, defined as follows:

$$D_{\text{cham}}(\mathcal{S}_1, \mathcal{S}_2) = \frac{1}{2}\left(\frac{1}{|\mathcal{S}_1|}\sum_{p\in\mathcal{S}_1}\min_{q\in\mathcal{S}_2}||p,q||_2 + \frac{1}{|\mathcal{S}_2|}\sum_{p\in\mathcal{S}_2}\min_{q\in\mathcal{S}_1}||q,p||_2\right) \tag{12}$$

where $\mathcal{S}_1$ and $\mathcal{S}_2$ represent different two sets. p and q are the elements in $\mathcal{S}_1$ and $\mathcal{S}_2$, respectively. In our experiments, we use chamfer distance to calculate the distance between a prediction and a ground-truth polyline set. Each polyline set is represented by uniformly sampling a polyline to N_{pts} vertices, where N_{pts} is set to 20 in our experiments. The mAP is calculated as the average of chamfer AP in three-pixel distance thresholds, $\tau_{\text{chamfer}} \in \{2\text{pix}, 4\text{pix}, 8\text{pix}\}$. A prediction is considered a true-positive (TP) when the distance is below the given threshold. In our experiments, two categories are considered: lane and centerline, and their mAP metrics are abbreviated as AP_{lane} and AP_{cline}.

4) Experimental Settings: All the experiments are run on two GeForce GTX 3090 GPUs with PyTorch implementation. In training, we crop images to 768 × 768 patches for the Aerial Argoverse2 Dataset, and 1024 × 1024 patches for the CARLA Simulator Dataset. We use ResNet50 as the backbone for our networks with pretrained weights downloaded from the PyTorch library. The model trains 300 epochs on the Aerial Argoverse2 Dataset and 200 epochs on the CARLA Simulator Dataset. We set the feature dimension for the instance embedding head to 32. We reduce the learning rate by using the "poly" learning rate policy, in which the initial learning rate is multiplied by $(1 - (\text{iter}/\text{max_iter}))^{\text{power}}$ and power = 0.9. iter and max_iter are the current iteration and the maximum iteration, respectively. Random crops and horizontal flips are also applied.

TABLE II
COMPARISON WITH DIFFERENT METHODS ON THE AERIAL ARGOVERSE2 DATASET

Method	Backbone	AP_{lane}	AP_{cline}	mAP	FPS
HDMapNet [12]	ResNet50	23.58	22.68	23.13	0.42
He et al. [9]	ResNet50	23.67	22.90	23.29	0.53
PivotNet [27]	ResNet50	31.22	27.72	29.47	24.43
SteamMapNet [26]	ResNet50	33.77	28.63	31.20	23.82
VectorMapNet [13]	ResNet50	35.79	32.15	33.97	23.28
MapTR [14]	ResNet50	36.71	33.17	34.94	27.31
AerialHDMapper(Ours)	ResNet50	<u>41.92</u>	<u>38.18</u>	<u>40.05</u>	25.15
AerialHDMapper(Ours)	ResNet101	**46.09**	**42.82**	**44.45**	20.57

B. Main Results

1) Aerial Argpverse2 Dataset: As shown in Table II, we compare AerialHDMapper with SOTA methods on the Aerial Argoverse2 Dataset. To demonstrate the effectiveness of our proposed method, we included several other methods for comparison, including those based on aerial imagery [9] and onboard images [12], [13], [14], [26], [27]. The results indicate that our approach outperforms previous methods, achieving the best overall performance. Specifically, AerialHDMapper shows a significant improvement over the aerial imagery-based method proposed by He and Balakrishnan [9], with an increase of 16.76 mAP under the same ResNet50 settings. Compared to MapTR [14], the current SOTA method for onboard image-based HDMap construction, our ResNet50-based approach achieves an improvement of 5.11 mAP. It is worth noting that although SteamMapNet [26] and PivotNet [27] are the latest methods at present, they are not suitable for aerial imagery because their framework is designed for on-board images, resulting in their poor performance in the Aerial Argoverse2 Dataset. To further explore the potential of AerialHDMapper, we also experimented with the ResNet101 backbone. This configuration brought an additional improvement of 4.40 mAP, demonstrating the scalability and robustness of our method. Furthermore, the visual results of AerialHDMapper in several aerial imagery are shown in Fig. 4.

Fig. 4 shows visual comparisons between AerialHDMapper and other mainstream methods on the Aerial Argoverse2 Dataset. It can be observed that AerialHDMapper accurately predicts lane centerlines at corners and intersections, and the topological relationships between map elements are more reasonable. In addition, the blue dashed ellipses emphasize the prediction results of different methods at occlusions. It can be seen that AerialHDMapper predicts map elements at occlusions better than other methods. This is due to the fact that the DirAttn module uses the information around the map element to complement the current location.

2) CARLA Simulator Dataset: To further verify the effectiveness of AerialHDMapper, we conducted comparative

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

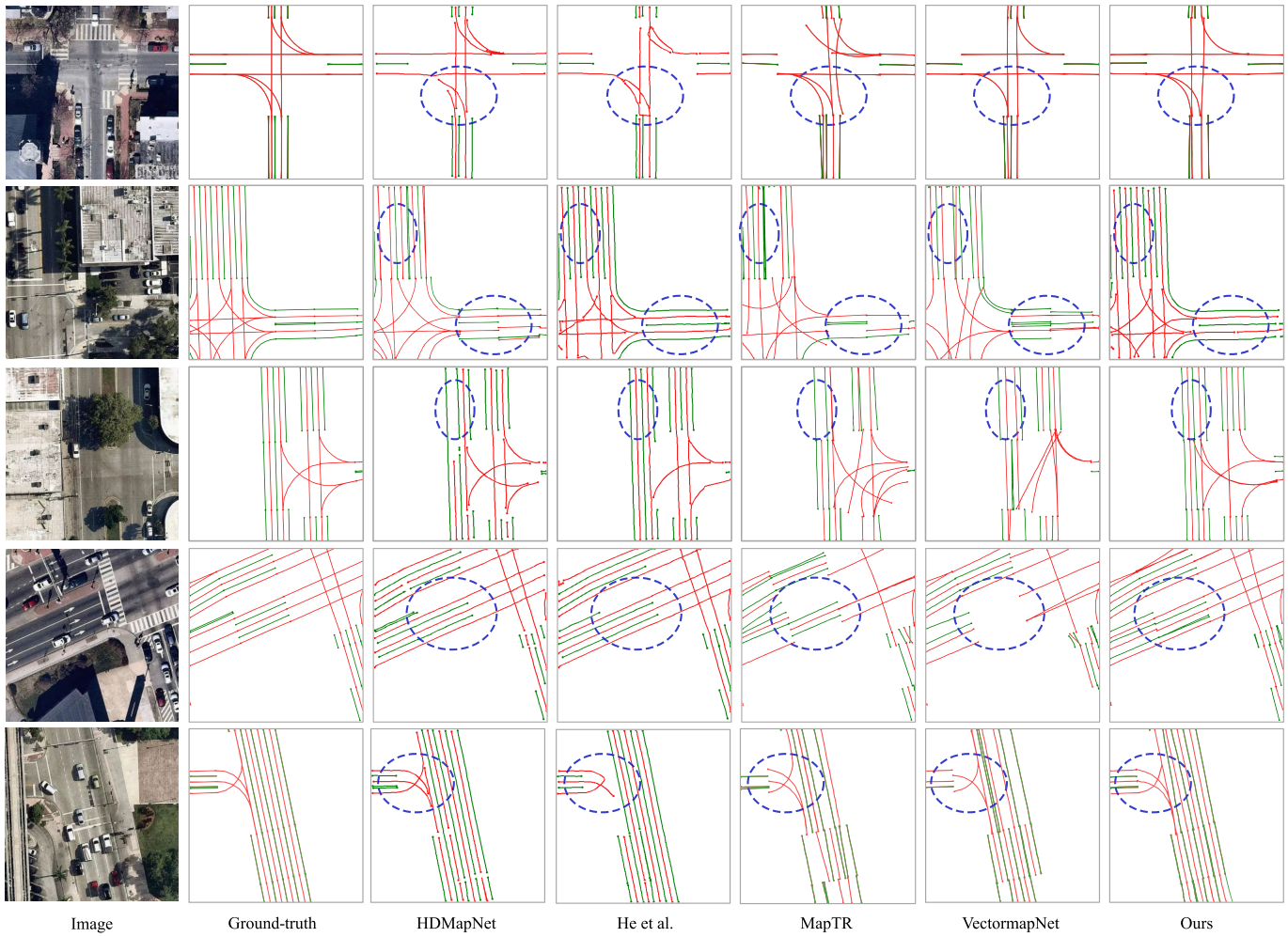


Fig. 4. Visual comparisons on the Aerial Argoverse2 Dataset. From left to right: image, ground truth, HDMapNet [12], He and Balakrishnan [9], MapTR [14], VectorMapNet [13], and ours AerialHDMapper.

TABLE III
COMPARISON WITH DIFFERENT METHODS ON THE CARLA SIMULATOR DATASET

Method	Backbone	AP_{lane}	AP_{cline}	mAP	FPS
HDMapNet [12]	ResNet50	42.44	39.77	41.11	0.64
He et al. [9]	ResNet50	52.12	48.74	50.43	1.16
PivotNet [27]	ResNet50	59.45	57.21	58.33	18.02
SteamMapNet [26]	ResNet50	57.10	58.39	57.75	16.54
VectorMapNet [13]	ResNet50	63.24	63.04	63.14	17.91
MapTR [14]	ResNet50	64.70	62.69	63.70	17.84
AerialHDMapper(Ours)	ResNet50	**65.97**	63.66	64.81	15.21
AerialHDMapper(Ours)	ResNet101	64.88	**65.84**	**65.36**	13.26

experiments using the CARLA Simulator Dataset. The results of these experiments are presented in Table III, demonstrating that our method consistently outperforms other SOTA methods. Specifically, when compared to MapTR [14], AerialHDMapper with the ResNet50 backbone achieves an improvement of 1.11 mAP. In addition, replacing the ResNet50 backbone with ResNet101 results in a modest improvement of 0.55 mAP. This increase is less significant than the improvement observed on the Aerial Argoverse2 Dataset. This phenomenon may be attributed to the relatively simple scenarios simulated in the CARLA Simulator Dataset, where a less complex model can still achieve good performance.

Notably, although the CARLA Simulator Dataset is larger in scale, segmentation-based methods achieve faster prediction speeds on the CARLA Simulator Dataset compared to the Aerial Argoverse2 Dataset. This is because the lane structures in the CARLA Simulator Dataset are simpler, leading to more accurate segmentation predictions and consequently shorter postprocessing times. To investigate this, we recorded the average prediction and postprocessing times for segmentation-based methods on both datasets. The results show negligible differences in prediction time; however, the average postprocessing time on the CARLA Simulator Dataset is 2.15 s, compared to 1.73 s on the Aerial Argoverse2 Dataset.

Fig. 5 illustrates visual comparisons between AerialHDMapper and other mainstream methods on the CARLA Simulator Dataset. The blue dashed boxes emphasize the prediction results of the different methods at intersections. When different map elements intersect, AerialHDMapper predicts the elements more accurately compared to segmentation-based methods (HDMapNet [12] and He and Balakrishnan [9]).

It is worth noting that the visualization results of AerialHDMapper are not significantly different from those of MapTR [14] and VectorMapNet [13], demonstrating that

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

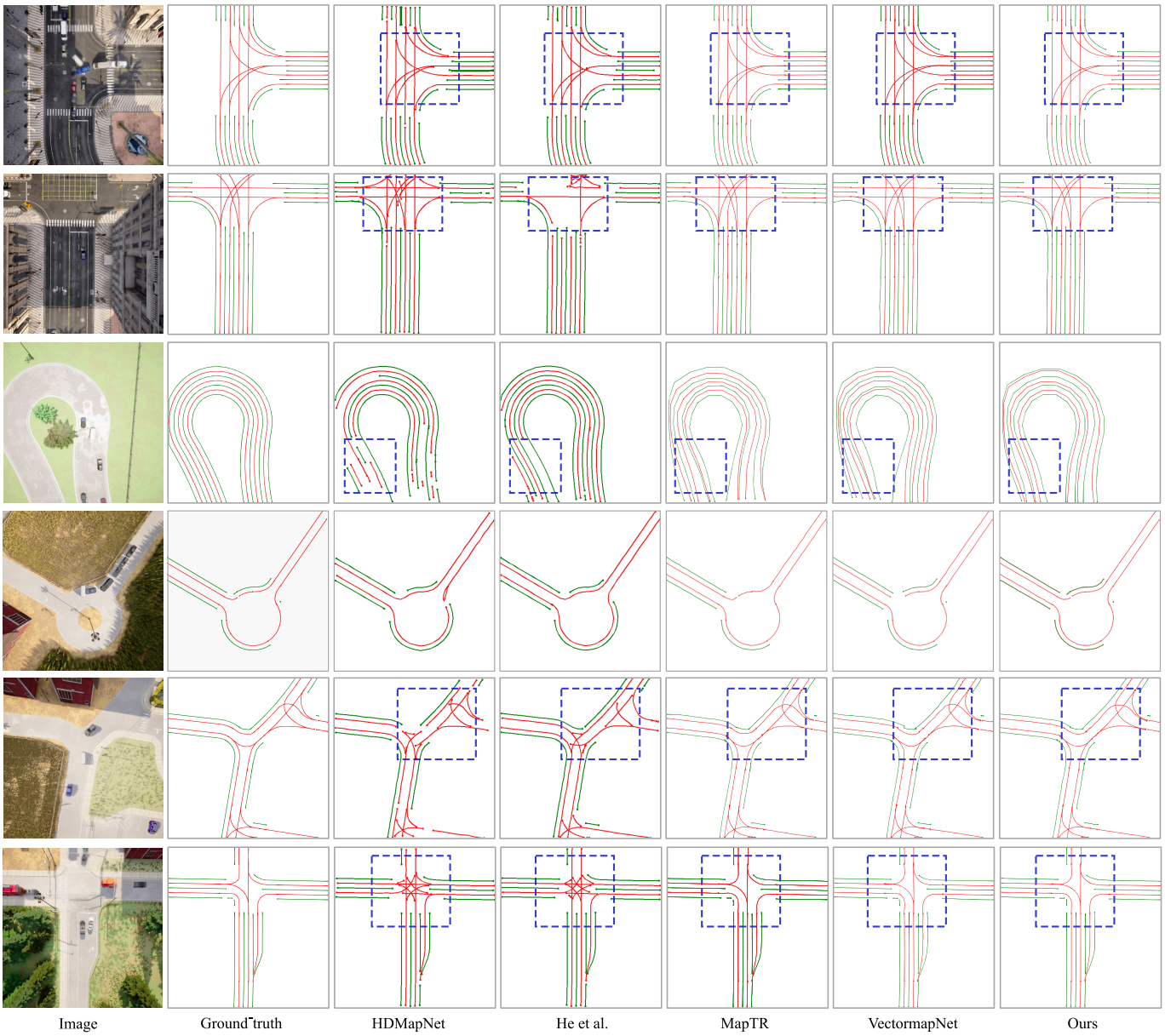


Fig. 5. Visual comparisons on the CARLA Simulator Dataset. From left to right: image, ground truth, HDMapNet [12], He and Balakrishnan [9], MapTR [14], VectormapNet [13], and ours AerialHDMapper.

predicting polylines is more suitable for lane-level HDMap construction than segmentation-based methods.

In addition, in the third row of Fig. 5, we observe that in very curvy scenarios, directly predicting vectors for long objects can lead to noticeable coarse polyline artifacts due to an insufficient number of preset points. In contrast, as shown in the fourth row, although it is also a very curvy scenario, dividing objects into multiple segments results in significantly improved visualization compared to the third row. The impact of the number of preset points on the method will be discussed in detail in the ablation study.

C. Ablation Studies

In this section, ablation experiments are conducted to examine the effectiveness of our proposed modules and designs. For fair comparisons, all experiments are conducted on the Aerial Argoverse2 Dataset with 300 epochs for training. ResNet50 is employed as the image backbone.

In Table IV, we provide ablations to verify the designing merits of our proposed modules: the baseline model, the DirAttn module, and the SemSeg. The baseline model achieves an AP_{lane} of 37.07, AP_{cline} of 33.38, and an mAP of 35.23. Our baseline model achieves a mAP boost of 0.29 over the previous best MapTR model. With the gradual addition of DirAttn modules, when DirAttn modules are added at three layers ($1 + 2 + 3$), the model achieves an AP_{lane} of 41.45, AP_{cline} of 36.54, and an mAP of 38.99. Finally, combining DirAttn modules at three layers with SemSeg yields the best performance, with an AP_{lane} of 41.92, AP_{cline} of 38.18, and an mAP of 40.05. This demonstrates the effectiveness of incorporating both the DirAttn module and SemSeg in enhancing HDMap extraction.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

ABLATION STUDY OF DIFFERENT COMPONENTS: BASELINE, DIRATTN MODULE, SEMSEG

Baseline	DirAttn	SemSeg	AP_{lane}	AP_{cline}	mAP	FPS	Params
✓			37.07	33.38	35.23	28.93	46.15M
✓	1		39.08	35.77	37.43	27.12	51.56M
✓	1+2		40.36	35.88	38.12	26.19	54.39M
✓	1+2+3		41.45	36.54	38.99	25.73	56.07M
✓	1+2+3	✓	**41.92**	**38.18**	**40.05**	25.15	56.95M

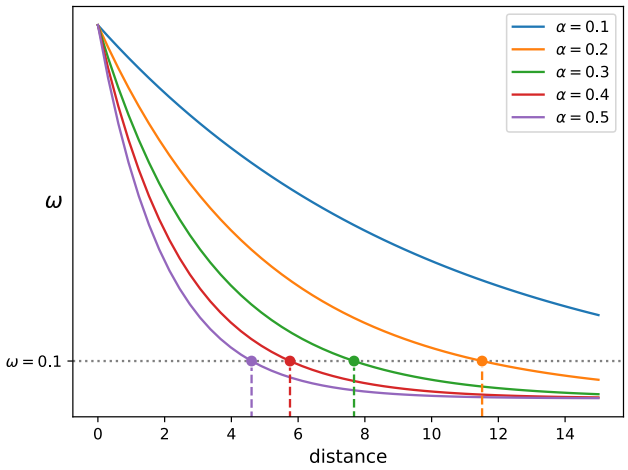


Fig. 6. Effect of α selections to ω of the DirAttn module. The higher value of α reduces the weight of distant locations in ω.

TABLE V

COMPARISON WITH DIFFERENT ATTENTION MECHANISMS ON THE AERIAL ARGOVERSE2 DATASET

Mathod	AP_{lane}	AP_{cline}	mAP	FPS	Params
CrissCross [39]	38.64	35.75	37.20	25.48	53.84M
DeformAttn [33]	38.87	36.49	37.68	21.49	53.02M
DualAttn [40]	39.69	37.54	38.62	21.80	82.50M
Non-local [41]	**41.99**	38.05	40.02	14.59	89.15M
DirAttn	41.92	**38.18**	**40.05**	25.15	56.95M

Notably, adding the SemSeg branch brings a greater improvement to centerlines than to lanes. We speculate that this is because the SemSeg branch enhances the network's understanding of the road scene beyond explicit visual features, enabling effective detection of unmarked centerlines through contextual and semantic information. Since visible lanes already provide strong visual cues detectable without additional assistance, adding the segmentation branch yields a more significant improvement for unmarked centerlines.

In addition, we observe that the baseline model achieves an excellent frames per second (FPS) of 28.93. This is due to the efficiency boost brought about by the end-to-end framework we designed. Introducing the DirAttn and SemSeg modules increases the model's parameters from 46.15 to 56.95 million and decreases the FPS from 28.93 to 25.15. Despite the increased parameter count and reduced FPS, the model's accuracy metrics show significant improvement. This indicates that incorporating the DirAttn and SemSeg modules enhances detection accuracy at the expense of some inference speed.

1) DirAttn Module: In order to verify the effectiveness of the DirAttn module, we first compare it with the mainstream attention mechanisms, and then explore the parameter Settings of the DirAttn module.

Table V shows that the DirAttn module achieves the highest mAP of 40.05 and AP_{cline} of 38.18, outperforming other methods in accuracy. It also maintains competitive efficiency with 25.15 FPS, significantly faster than the nonlocal attention mechanism (nonlocal) [41] of 14.59 FPS, while having a moderate parameter count of 56.95 M, much lower than the nonlocal of 89.15 M and the Dual Attention (DualAttn) mechanism [40] of 82.50 M. These results demonstrate that the DirAttn module achieves the best balance between accuracy and efficiency, making it highly effective for HD map construction.

According to 1, the weight $\omega_{i,j}$ is inversely related with the distance $L_{i,j}$. The influence of $L_{i,j}$ is regulated by the temperature parameter α and the size of patch regions D. As α increases, the weight $\omega_{i,j}$ assigned to distant pixels diminishes. To access the effect of α and D on network performance, we conduct a series of ablation experiments. For clarity, we simplify the notation by referring to $\omega_{i,j}$ as ω in the following descriptions.

First, the relationship between ω and the distance for different α is constructed, as illustrated in Fig. 6. Based on this relationship, the corresponding patch region size D can be found for different α. In particular, we set a threshold of $\omega = 0.1$, ignoring pixels where $w < 0.1$. This setting allows us to determine the effective patch region size D. Various combinations of α and D are presented in Table VI. Notably, when $\alpha = 0.1$, the corresponding D becomes excessively large, significantly reducing network efficiency. To address this, we restrict D to a maximum of 15 when $\alpha = 0.1$.

As shown in Table VI, when $\alpha = 0.1$ and $D = 15$, the model achieves an AP_{lane} of 43.51, AP_{cline} of 39.97, and an mAP of 41.74, with an slow FPS of 20.87. The best overall performance is observed with $\alpha = 0.2$ and $D = 11$, yielding an AP_{lane} of 41.92, AP_{cline} of 38.18, an mAP of 40.05, and an better FPS of 25.15. Further increasing α to 0.4 and 0.5 with $D = 5$ results in a drop in mAP to 36.72 and 37.14, respectively, but with a marginally higher FPS of around 28.5. This demonstrates that a balanced setting of $\alpha = 0.2$ and $D = 11$ is optimal for achieving the best performance.

To further explore the principle of the DirAttn module, we visualize ω after the first layer in Fig. 7. The visualization result shows that assigns greater attention to positions along the map element direction and its perpendicular direction. It is worth noting that for some easy-to-predict map elements, the ω is not exactly along the direction of the map element. This may be because when the DirAttn module predicts the direction of the current position, no loss function supervision is set for the prediction results.

2) Polyline Generator: Finally, we investigate the ablation experiment on the setting of the polyline generator. In our experiment settings, we compare different selections of the number of queries N_q and the number of points in each query N_p. The number of queries ranges from 100 to 250, and the number of points is set from 10 to 40. As illustrated in Table VII, when $N_p = 20$, increasing N_q from 100 to 200 improves performance, with mAP peaking at 40.05 and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

PERFORMANCE OF THE DIRATTN MODULE DESIGN WITH DIFFERENT SETTING OF α AND SIZE OF PATCH REGION D

α	D	AP_{lane}	AP_{cline}	mAP	FPS
0.1	15	43.51	39.97	41.74	20.87
0.2	11	41.92	38.18	40.05	25.15
0.3	7	41.92	37.08	39.50	27.93
0.4	5	38.55	34.89	36.72	28.52
0.5	5	39.52	34.75	37.14	28.59



Fig. 7. Visualization of ω after the first layer of the encoder. ω assigns greater attention to positions along the map element direction and its perpendicular direction.

TABLE VII

PERFORMANCE OF THE POLYLINE GENERATOR DESIGN WITH DIFFERENT NUMBER OF QUERIES N_q AND POINTS N_p

N_q	N_p	AP_{lane}	AP_{cline}	mAP	FPS
100	20	31.66	24.12	27.89	**28.38**
150	20	34.73	29.46	32.10	26.12
200	**20**	**41.92**	**38.18**	**40.05**	25.15
250	20	39.33	35.21	37.27	22.94
200	10	40.63	36.45	38.54	**27.08**
200	**20**	**41.92**	**38.18**	**40.05**	25.15
200	30	40.87	36.87	38.87	22.45
200	40	35.56	30.17	32.87	18.29

FPS at 25.15; however, at $N_q = 250$, mAP slightly decreases to 37.27 and FPS drops to 22.94. For a fixed $N_q = 200$, increasing the number of N_p does not always lead to performance improvement. When $N_p = 20$, the mAP reaches its optimal value, and as the number of N_p continues to increase, both mAP and FPS decrease.

To investigate the impact of different values of N_p on AerialHDMapper, we present qualitative results in Fig. 8. In the first row, we showcase the influence of varying N_p on predictions in highly curved scenarios. As the number of points increases, coarse polyline artifacts gradually diminish. This improvement occurs because curves typically consist of multiple small segments with complex local variations. A higher predefined point count allows the network to use more points to capture the intricate details of the curves when generating vectors. The second and third rows demonstrate the impact of different N_p on predictions in straight-line scenarios. For better visualization, The blue dashed boxes in the second row are enlarged in the third row. When $N_p = 10$, the lines are smooth, but there is a noticeable offset at the connection points between elements. On the other hand, when N_p is set too high,

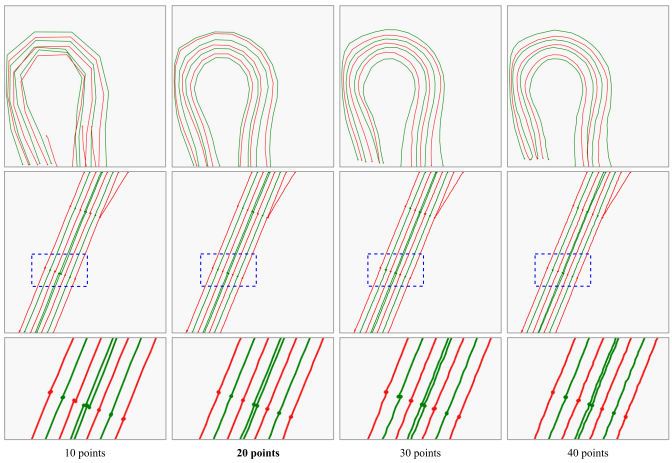


Fig. 8. Effect of N_p selections to AerialHDMapper. Too high or too low a value of N_p will have a negative impact on the network prediction.

TABLE VIII

COMPARISON WITH MAPTR ON THE LARGE-SCALE DATASET

Method	AP_{lane}	AP_{cline}	mAP	FPS	Params	Total
MapTR [14]	32.69	30.98	31.84	28.91	35.28M	10.93
AerialHDMapper(Ours)	35.19	32.46	33.82	25.62	56.95M	12.34

internal points exhibit noise, causing the polylines to appear jagged. This phenomenon can be explained by the spacing between points: if N_p is too small, the large spacing makes it difficult for the polyline generator to accurately track the lines, resulting in offsets. Conversely, with larger N_p values, the increased point redundancy introduces greater prediction complexity, resulting in noise within the generated lines.

By comparing the impact of different values of N_p, we choose $N_p = 20$ as the preset value of the experiment, because its performance is relatively stable in different scenarios.

3) Large-Scale Dataset Experiments: To evaluate the generalization ability and runtime efficiency of our proposed method, we conducted experiments on a large-scale dataset. This dataset was collected from the region surrounding the Aerial Argoverse2 Dataset, with the image size of 11 470 × 4347. We first split the dataset into 768 × 768 patches, performed predictions on each patch, and then stitched the predicted vectors to obtain the final results.

Table VIII compares the accuracy and runtime efficiency of MapTR and ours AerialHDMapper. AerialHDMapper outperforms MapTR in terms of accuracy, with AP_{lane}, AP_{cline}, and mAP increasing by 2.5, 1.48, and 1.98, respectively. These improvements demonstrate AerialHDMapper's superior performance in the HDMap construction task. However, this enhanced accuracy comes at the cost of runtime efficiency, as AerialHDMapper has a lower FPS, uses more parameters, and requires more total runtime compared to MapTR. These results highlight AerialHDMapper's superior accuracy at the cost of slightly reduced runtime efficiency.

Fig. 9 illustrates visual comparisons between AerialHDMapper and MapTR on the large-scale dataset. The results show that AerialHDMapper produces more accurate vector

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

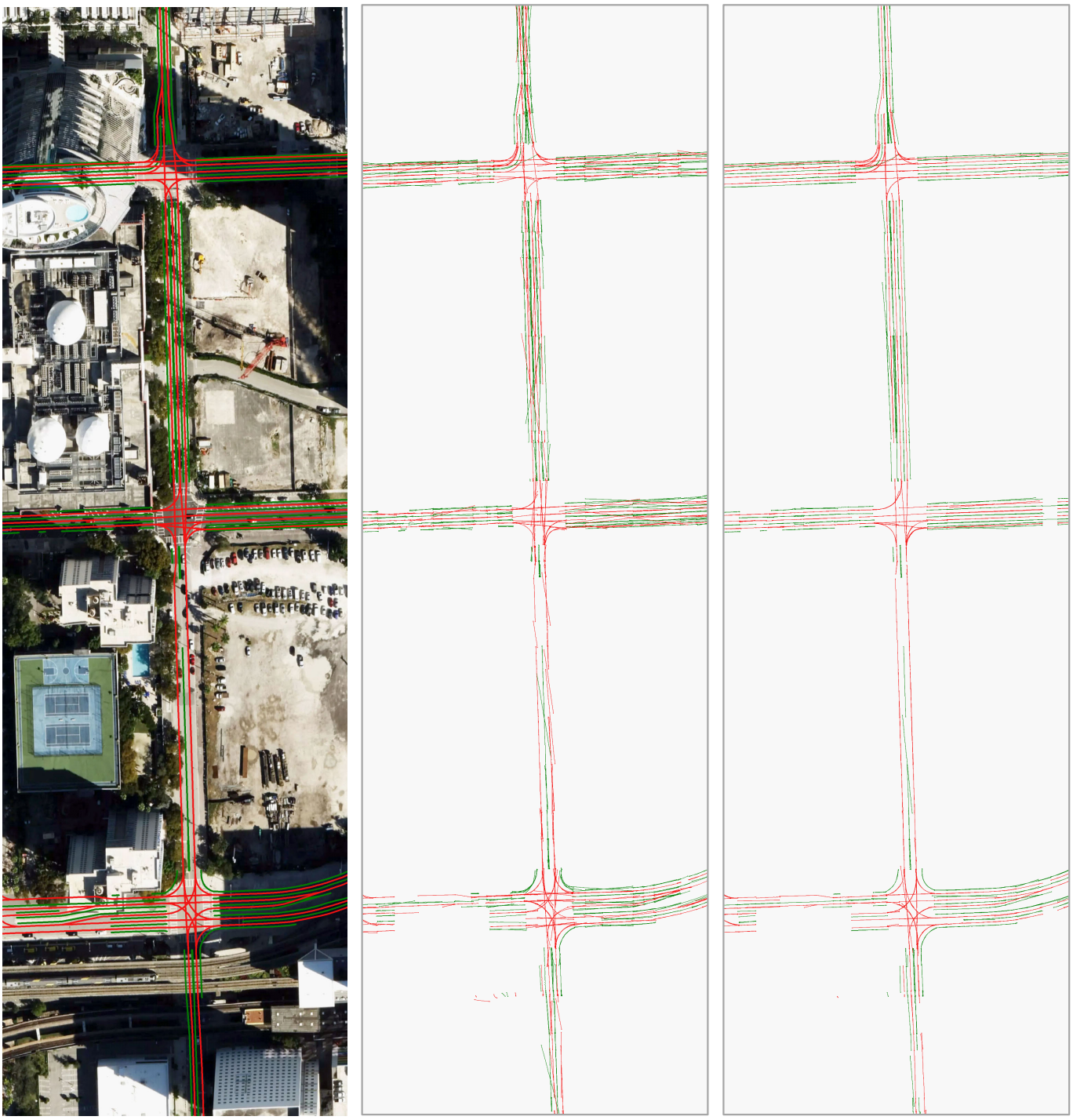


Fig. 9. Visual comparisons on the large-scale dataset. From left to right: image and ground truth, MapTR [14], and ours AerialHDMapper.

predictions compared to MapTR. Specifically, the predicted vectors exhibit fewer artifacts and less noise, contributing to smoother and more continuous line representations. This improvement is especially noticeable in complex scenarios where precise lane and centerline extraction is critical.

V. Conclusion

This article introduces AerialHDMapper, a transformer-based learning framework designed for HDMap construction from aerial imagery. AerialHDMapper comprises two primary components: the image encoder and the polyline generator, with an additional SemSeg to assist in prediction. We integrate the DirAttn module between each layer of the image encoder. The rationale behind the DirAttn module is that HDMap elements, even when occluded or not visually represented, can be inferred using information along their tangential and orthogonal directions. By performing attention operations on image features in these directions, we can enhance the current

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.

positional information. The polyline generator, adapted from DETR, is capable of identifying long-sequence irregular map elements, thereby speeding up the prediction process, eliminating the need for complex postprocessing, and enabling object-level prediction.

For future work, it would be valuable to integrate traffic rules with HDMap construction. Another promising direction for HDMap construction from aerial imagery is to focus on constructing HDMaps from large-scale aerial imagery. Predictions from small-scale images contain less information and are less efficient. Thus, developing methods to quickly predict large-scale images while ensuring the completeness of the prediction results will be a highly promising avenue for future research.

REFERENCES

[1] J. Ziegler et al., "Making Bertha drive—An autonomous journey on a historic route," *IEEE Intell. Transp. Syst. Mag.*, vol. 6, no. 2, pp. 8–20, Summer. 2014.

[2] R. Matthaei and M. Maurer, "Autonomous driving—A top-down-approach," *At Automatisierungstechnik*, vol. 63, no. 3, pp. 155–167, Mar. 2015.

[3] S. Ulbrich et al., "Towards a functional system architecture for automated vehicles," 2017, *arXiv:1703.08557*.

[4] X. Xiong, Y. Liu, T. Yuan, Y. Wang, Y. Wang, and H. Zhao, "Neural map prior for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 17535–17544.

[5] J. Zhang and S. Singh, "LOAM: LiDAR odometry and mapping in real-time," in *Proc. Robot., Sci. Syst.*, Berkeley, CA, USA, 2014, vol. 2, no. 9, pp. 1–9.

[6] T. Shan and B. Englot, "LeGO-LOAM: Lightweight and ground-optimized LiDAR odometry and mapping on variable terrain," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 4758–4765.

[7] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "LIO-SAM: Tightly-coupled LiDAR inertial odometry via smoothing and mapping," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 5135–5142.

[8] R. Liu, J. Wang, and B. Zhang, "High definition map for automated driving: Overview and analysis," *J. Navig.*, vol. 73, no. 2, pp. 324–341, 2020.

[9] S. He and H. Balakrishnan, "Lane-level street map extraction from aerial imagery," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2022, pp. 1496–1505.

[10] F. Bastani et al., "RoadTracer: Automatic extraction of road networks from aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4720–4728.

[11] G. Mattyus, W. Luo, and R. Urtasun, "DeepRoadMapper: Extracting road topology from aerial images," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 3438–3446.

[12] Q. Li, Y. Wang, Y. Wang, and H. Zhao, "HDMapNet: An online HD map construction and evaluation framework," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 4628–4634.

[13] Y. Liu, Y. Wang, Y. Wang, and H. Zhao, "VectorMapNet: End-to-end vectorized HD map learning," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2022, pp. 22352–22369.

[14] X. Liu, S. Wang, W. Li, R. Yang, J. Chen, and J. Zhu, "MGMap: Mask-guided learning for online vectorized HD map construction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 14812–14821.

[15] L. Ma, Y. Li, J. Li, Z. Zhong, and M. A. Chapman, "Generation of horizontally curved driving lines in HD maps using mobile laser scanning point clouds," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 5, pp. 1572–1586, May 2019.

[16] A. Batra, S. Singh, G. Pang, S. Basu, C. V. Jawahar, and M. Paluri, "Improved road connectivity by joint learning of orientation and segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10385–10393.

[17] V. Mnih and G. E. Hinton, "Learning to detect roads in high-resolution aerial images," in *Proc. Eur. Conf. Comput. Vis.*, 2010, pp. 210–223.

[18] G. Cheng, Y. Wang, S. Xu, H. Wang, S. Xiang, and C. Pan, "Automatic road detection and centerline extraction via cascaded end-to-end convolutional neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 6, pp. 3322–3337, Jun. 2017.

[19] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Commun. ACM*, vol. 27, no. 3, pp. 236–239, Mar. 1984.

[20] N. Homayounfar, J. Liang, W.-C. Ma, J. Fan, X. Wu, and R. Urtasun, "DAGMapper: Learning to map by discovering lane topology," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2019, pp. 2911–2920.

[21] Z. Xu, Y. Sun, and M. Liu, "ICurb: Imitation learning-based detection of road curbs using aerial images for autonomous driving," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 1097–1104, Apr. 2021.

[22] Y. Yu et al., "MarkCapsNet: Road marking extraction from aerial images using self-attention-guided capsule network," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[23] Z. Xu et al., "RNGDet: Road network graph detection by transformer in aerial images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4707612.

[24] Z. Li et al., "BEVFormer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers," in *Proc. ECCV*, vol. 13669, 2022, pp. 1–18.

[25] Z. Qin, J. Chen, C. Chen, X. Chen, and X. Li, "UniFusion: Unified multi-view fusion transformer for spatial–temporal representation in bird's-eye-view," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 8690–8699.

[26] T. Yuan, Y. Liu, Y. Wang, Y. Wang, and H. Zhao, "StreamMapNet: Streaming mapping network for vectorized online HD map construction," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2024, pp. 7341–7350.

[27] W. Ding, L. Qiao, X. Qiu, and C. Zhang, "PivotNet: Vectorized pivot learning for end-to-end HD map construction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 3672–3682.

[28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, vol. 16, Sep. 2016, pp. 770–778.

[29] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2020, pp. 213–229.

[30] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annu. Conf. Robot Learn.*, vol. 78, S. Levine, V. Vanhoucke, and K. Goldberg, Eds., Nov. 2017, pp. 1–16.

[31] B. Wilson et al., "Argoverse 2: Next generation datasets for self-driving perception and forecasting," *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks (NeurIPS Datasets Benchmarks)*, Aug. 2021.

[32] Z. Xu, Y. Liu, Y. Sun, M. Liu, and L. Wang, "RNGDet++: Road network graph detection by transformer with instance segmentation and multi-scale features enhancement," *IEEE Robot. Autom. Lett.*, vol. 8, no. 5, pp. 2991–2998, May 2023.

[33] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," 2020, *arXiv:2010.04159*.

[34] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logistics Quart.*, vol. 2, pp. 83–97, Mar. 1955.

[35] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.

[36] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1440–1448.

[37] M. H. Ng, K. Radia, J. Chen, D. Wang, I. Gog, and J. E. Gonzalez, "BEV-seg: Bird's eye view semantic segmentation using geometry and semantic point cloud," 2020, *arXiv:2006.11436*.

[38] B. Pan, J. Sun, H. Y. T. Leung, A. Andonian, and B. Zhou, "Cross-view semantic segmentation for sensing surroundings," *IEEE Robot. Autom. Lett.*, vol. 5, no. 3, pp. 4867–4873, Jul. 2020.

[39] Z. Huang, X. Wang, L. Huang, C. Huang, Y. Wei, and W. Liu, "CCNet: Criss-cross attention for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 603–612.

[40] J. Fu et al., "Dual attention network for scene segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. pattern Recognit.*, Jun. 2019, pp. 3141–3149.

[41] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7794–7803.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:52:27 UTC from IEEE Xplore. Restrictions apply.